

Practical Week 11

Task 1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Step 1: Create a C file

nano counter.c

```
GNU nano 7.2 counter.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_create(&t3, NULL, increment, NULL);
    pthread_create(&t4, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Final counter value: %d\n", counter);

    return 0;
}
```

Step 2 : Compile and run

```
bhavan@DESKTOP-BLT5L3R:~$ cd ~/OSSP
mkdir Week11
cd Week11
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ nano counter.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ gcc counter.c -o counter -pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter
Final counter value: 221313
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter
./counter
./counter
Final counter value: 184885
Final counter value: 192090
Final counter value: 147001
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$
```

Step 3: Fix the Race Condition Using Mutex

nano counter_mutex.c

```
GNU nano 7.2 counter_mutex.c *
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2, t3, t4;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);
    pthread_create(&t3, NULL, increment, NULL);
    pthread_create(&t4, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    printf("Final counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Step 4: Compile and Run the Mutex Version

```
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ nano counter_mutex.c
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ gcc counter_mutex.c -o counter_mutex -pthread
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter_mutex
Final counter value: 400000
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter_mutex
./counter_mutex
./counter_mutex
Final counter value: 400000
Final counter value: 400000
Final counter value: 400000
```

Observation:

A multithreaded counter application was developed using POSIX threads. Multiple threads updated a shared counter concurrently, producing inconsistent values due to a race condition. A mutex was then used to synchronize access to the shared variable.

Result:

The race condition was successfully demonstrated, and the mutex-protected program produced the correct counter value of **400000** consistently.

Task 2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

Step 1: Run the race-condition version again

```
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter
Final counter value: 133271
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter
./counter
./counter
Final counter value: 172279
Final counter value: 210973
Final counter value: 165352
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ |
```

Step 2: Run the mutex version

```
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ ./counter_mutex
./counter_mutex
./counter_mutex
Final counter value: 400000
Final counter value: 400000
Final counter value: 400000
bhavan@DESKTOP-BLT5L3R:~/OSSP/Week11$ |
```

Observation:

The race-condition program produced inconsistent counter values because multiple threads accessed the shared variable concurrently. After adding pthread_mutex_lock() and pthread_mutex_unlock(), access to the counter was synchronized.

Result:

The mutex version consistently produced the expected value of **400000**, demonstrating that synchronization prevents race conditions and ensures correct shared-data updates. The trade-off is that mutex locking adds some execution overhead because threads must wait for access to the shared resource.

